

A GraphQL Bug Hunt That Ended Up Ignored

By J0o1ey · Translated from the Chinese original

0x00 How It Started

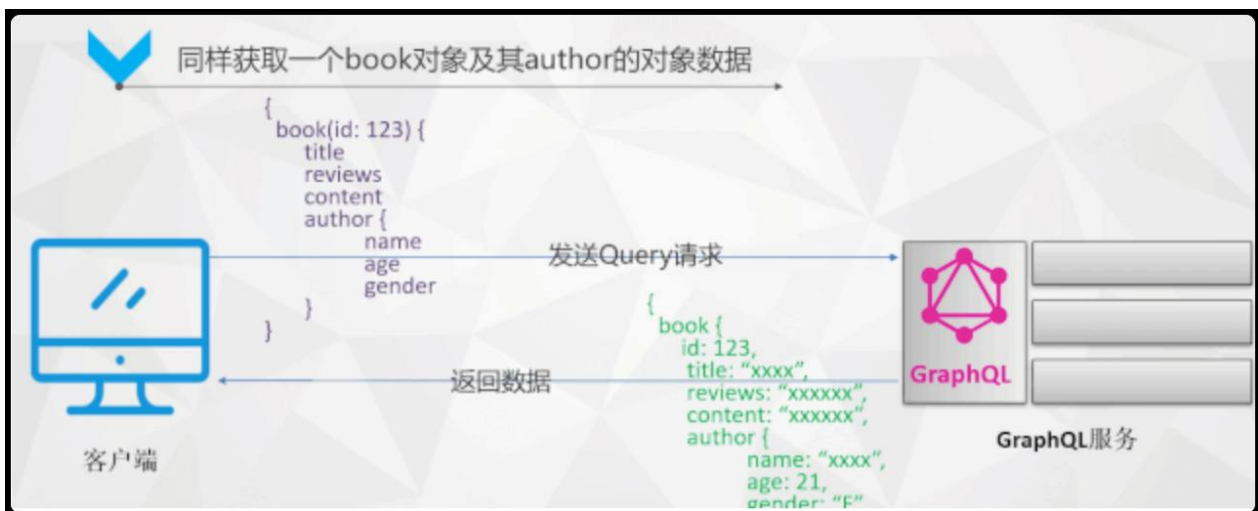
A few days ago I was hunting a high-bounty vendor on HackerOne. After walking through the business points, I happened to notice a GraphQL interface in the HTTP history.

And so began a prison-sentence of a journey.

0x01 Prerequisites: Introspection Queries

If GraphQL is new to you, take a look at the picture below.

In plain words: GraphQL is an open-source query language akin to RESTful APIs. It offers a clever new way to improve client-server interaction in applications, bringing fresh thinking to project construction and long-term maintenance.



In practice, I've found that not many domestic vendors use GraphQL — most are still RESTful — while it's rather popular abroad.

GraphQL supports a fairly special technique called the "introspection query".

Simply put, GraphQL ships with built-in "swagger-like" API documentation that you can retrieve via introspection: object definitions, interface parameters, and more.

When a user doesn't know which types in a GraphQL interface are available, they can ask via the `__schema` field which types exist.

For example, through `__schema introspection` you can learn more about the interface types. (The picture below is from an article by the Jiuxianqiao troop.)

```
1 {
2   schema {
3     types {
4       name
5     }
6   }
7 }

1 {
2   "data": {
3     "schema": {
4       "types": [
5         {
6           "name": "Root"
7         },
8         {
9           "name": "String"
10        },
11        {
12          "name": "Int"
13        },
14        {
15          "name": "FilmsConnection"
16        },
17        {
18          "name": "PageInfo"
19        },
20        {
21          "name": "Boolean"
22        }
23      ]
24    }
25  }
26 }
```

And introspection queries should generally never be directly reachable from the outside.

0x02 The Hunt

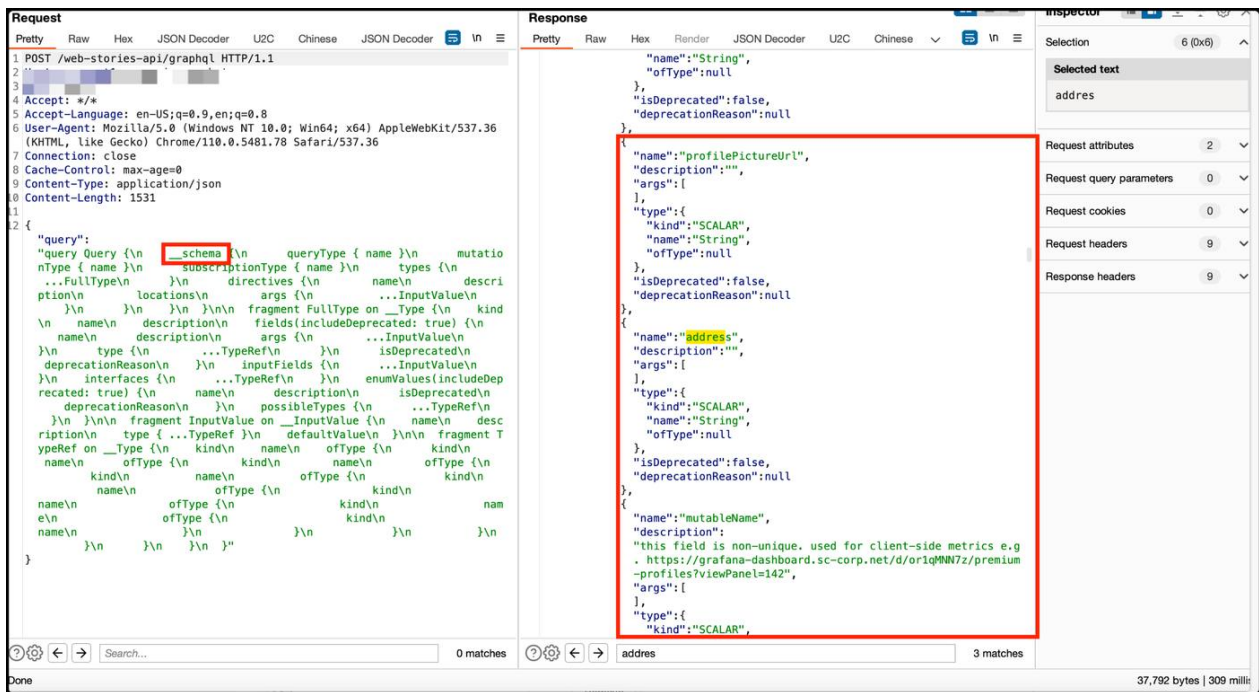
With the prerequisites covered, let's look at this GraphQL endpoint.

After capturing the vendor's GraphQL interface, I tried a `__schema` introspection query to pull extra information about it:

```
POST /web-stories-api/graphql HTTP/1.1
Host: xxxxxxxxxxxx
Accept-Encoding: gzip, deflate
Accept: */*
Accept-Language: en-US;q=0.9,en;q=0.8
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/110.0.5481.78 Safari/537.36
Connection: close
Cache-Control: max-age=0
Content-Type: application/json
Content-Length: 1531
{"query":"query Query {\n  __schema {\n    queryType { name }\n    mutationType { name }\n    subscriptionType { name }\n    types {\n      ...FullType\n    }\n    directives {\n      name\n      description\n      locations\n      args {\n        ...InputValue\n      }\n    }\n    fragment FullType on __Type {\n      kind\n      name\n      description\n      fields(includeDeprecated: true) {\n        name\n        description\n        args {\n          ...InputValue\n        }\n        type {\n          ...TypeRef\n        }\n        isDeprecated\n        deprecationReason\n      }\n      inputFields {\n        ...InputValue\n      }\n      interfaces {\n        ...TypeRef\n      }\n      enumValues(includeDeprecated: true) {\n        name\n        description\n        isDeprecated\n        deprecationReason\n      }\n      possibleTypes {\n        ...TypeRef\n      }\n    }\n    fragment InputValue on __InputValue {\n      name\n      description\n      type {\n        ...TypeRef\n      }\n      defaultValue\n    }\n    fragment TypeRef on __Type {\n      kind\n      name\n      ofType {\n        kind\n        name\n        ofType {\n          kind\n        }\n      }\n    }\n  }\n}\n"}

```

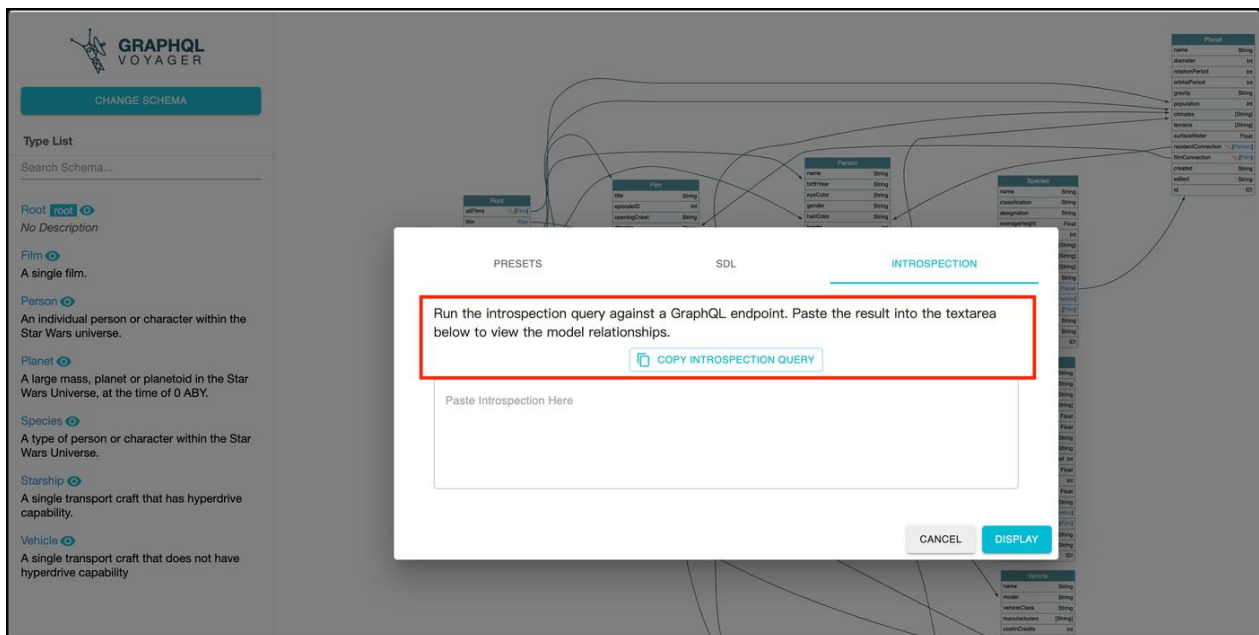
The result was very nice: the target returned the interface's full details.



But that blob was messy, so I decided to use a graphical introspection parser:

`https://apis.guru/graphql-voyager/`

This site provides a lovely graphical rendering for introspection JSON responses.



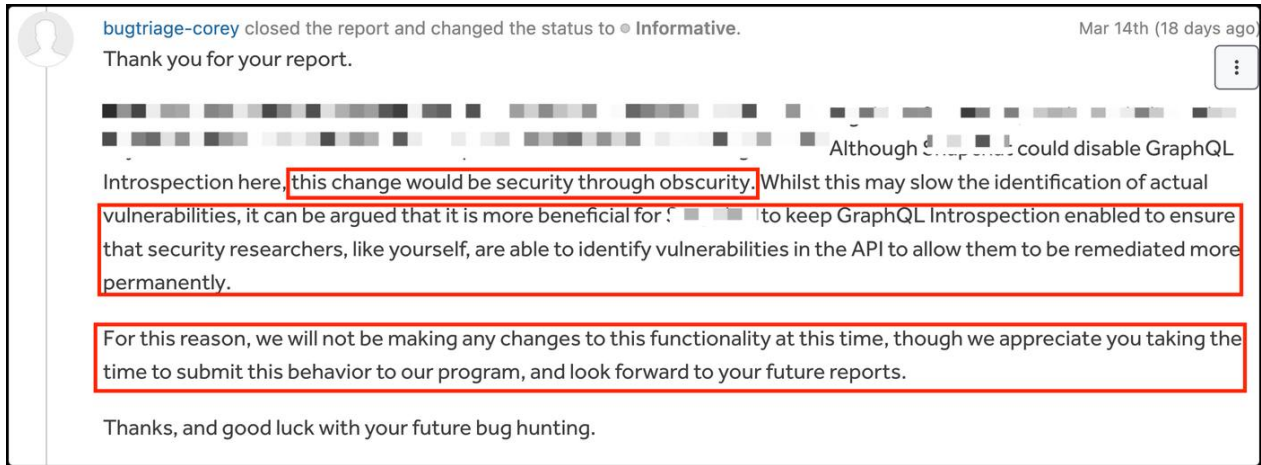
The parsed result (some fields involving the vendor's name are redacted):

I added the address parameter straight into the query — and out echoed the addresses attached to numerous "social shares".



Sadly, most addresses were null, or at best precise to the "town-level administrative region" — nothing that would pin down a poster's exact location.

Still, I decided to report it to the vendor's security team.



The vendor's replies, in essence: the address and similar fields, while not displayed in the public business requests, are not particularly sensitive information and would not affect user data security.

Although disabling introspection on this GraphQL interface would be safer, we believe keeping the introspection feature makes it easier for rookie security researchers (such as you) to find bugs in the API.

So we won't change this feature for now — though we do appreciate you spending time on our bounty program.

0x03 Afterword

Reading that, I was a little deflated — but it doesn't matter. In every bug hunt and every round of learning, what you walk away with (knowledge or experience) is what counts; the final outcome and bounty of a bug are usually heaven's decision, not ours to force.

And who's to say the next "introspection query" won't yield BOOM-grade information and vulnerabilities?

Hack to learn and learn to hack — you need both.